



SMART CONTRACT SECURITY AUDIT

ROYCO

SECURITY AUDIT REPORT

PREPARED BY Tomer Security First	CLIENT Royco Protocol
REPORT DATE July 2026	TOTAL FINDINGS 33 Findings
LANGUAGE Solidity 0.8.35	VERSION v1.0 - Final

SECTION 1

Tomer's Take

Royco tackles two different problems: protecting exposure to volatile assets, and incentivizing liquidity in DEX markets. What connects the two is a simple but powerful idea: only the parties who pay for an incentive receive its benefit.

Traditionally, when a protocol wants to attract liquidity, it pays users to deposit their assets. The problem is that the people funding the incentive are usually not the same people who directly benefit from the resulting liquidity. This means that someone external - typically the token issuer, the protocol, or another party somewhere along the way - must continuously spend money to keep the market attractive.

Royco changes this structure by making the benefit more targeted. Since only the parties paying for the incentive receive it, the same amount of capital can be used more efficiently. A smaller payment can have a much greater effect because it is not being distributed to participants who are not contributing to the underlying demand.

The protection mechanism is built around two layers: a junior tranche and a senior tranche. The senior tranche gives up part of its yield in exchange for protection, while the junior tranche earns that additional yield by agreeing to absorb the first losses in the event of slashing. Part of the senior tranche's yield can also be used to fund the liquidity needed for the associated markets.

One of the strongest aspects of the design is its modularity. The protection mechanism and the liquidity-incentive mechanism can work together, but they do not have to. A protocol can use one without adopting the other, which makes the system easier to integrate across different use cases.

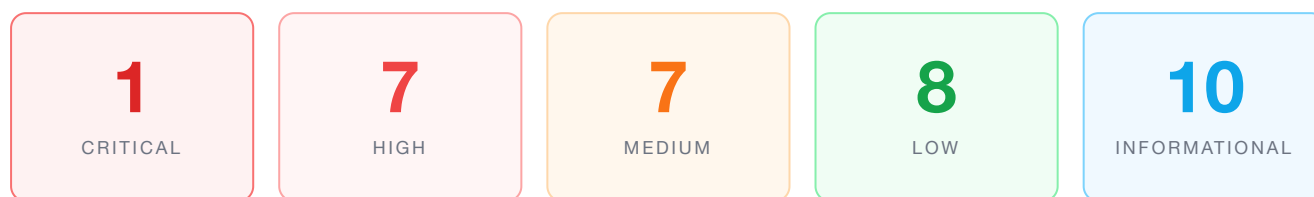
Another particularly interesting feature is how permissionless the system is. Anyone can deploy a new market with a single contract call. This turns Royco from a fixed product into infrastructure that other protocols and market participants can build on directly.

Ultimately, Royco is most useful when you have an opinion on what will happen next. If you believe an asset will perform well, the junior tranche or the liquidity side may offer the more attractive position. If you are more concerned about downside or slashing risk, the senior tranche provides a way to reduce that exposure.

Either way, if you have an opinion about the future of an asset and that asset is available through Royco, there is likely a position in the system that reflects that view.

SECTION 2

Executive Summary



Tomer Security First conducted a comprehensive security audit of the **Royco Day protocol** - a DeFi yield-tranching system that splits any yield source into three economic tranches: a capital-protected Senior Tranche (ST), a first-loss Junior Tranche (JT), and a market-making Liquidity Provider Tranche (LPT) powered by a Balancer V3 GyroECLP AMM. The audit covered the complete Solidity codebase, covering the accountant, entypoint queue, kernel orchestration, factory deployment templates, redemption/deposit logic, self-liquidation bonus mechanics, and Balancer V3 venue integration.

The audit identified **33 findings** across five severity tiers. **One critical-severity issue** was identified: an exploitable static fee vector in the Balancer V3 integration that enables profit extraction via mixed add/remove unlock sessions. **Seven high-severity issues** were identified: an arithmetic underflow in the LPT multi-asset redemption path, static fee imposition on external removers via hook-triggered reinvestment, LPT effective NAV overvaluation in the preview path, an LPT share price inflation attack at zero total supply, an LT protocol fee minted against idle ST premium shares that permanently dilutes LT `convertToAssets` whenever GyroECLP reinvestment is constrained, a YDM time-weighted average manipulation that converts a winning JT long into a forced liquidation when ST and JT assets are uncorrelated, and an incorrect deposit-size clamp in `_convertToShares` that causes value donation in normal markets and unbounded share inflation after a slash event.

Seven medium-severity issues were identified, covering an oracle-gated JT exit exploiting time-weighted share spread, pre-sync LPT rate sampling in the deposit reference path, a missing post-update safety check on minimum coverage governance, incorrect premium distribution after drawdown-recovery cycles, a liquidation state death spiral leading to irrecoverable market drain, pending liquidity premium omission in the LPT asset claims preview path, and a split-execution attack on the bootstrap deposit reference that causes the victim to forfeit nearly all deposited value to the protocol fee recipient. **Eight low-severity** and **ten informational** findings address rounding biases, access control role conflation, a missing NAV addend in asset claim scaling, NAV-based yield forfeiture asymmetries in the entry point, a post-op guard operator error in coverage utilization governance, partial execution rounding enabling incremental fee avoidance, unnecessary flexibility in the liquidity premium mint interface, a receiver blacklist bypass in the `ltRedeemMultiAsset` path when the venue returns zero ST shares, hardcoded `paysYieldFees: false` in Gyro ECLP token registration that renders the pool creator yield fee configuration permanently inert, a `disableUnbalancedLiquidity` configuration that silently breaks premium reinvestment, an asset mismatch in `_cancelDepositRequest` that enables EntryPoint drainage via factory config manipulation, a JT lockout in the FIXED_TERM observation period when no senior capital is present, and a JT deposit impermanent-loss dilution that transfers outstanding IL recovery rights to new depositors who bore none of the original coverage losses.

The Royco Day codebase demonstrates sophisticated financial engineering and thoughtful defensive design. The identified issues are concentrated in the complex interactions between the Balancer V3 integration and the protocol's bespoke accounting model - areas that benefit most from targeted review.

CONTENTS

Table of Contents

OVERVIEW

1	Tomer's Take	02
2	Executive Summary	03
3	Findings Overview	04
4	Protocol Overview	05
5	Audit Scope & Methodology	05
6	Audit Findings	06
7	Disclaimer	13

FINDINGS - CRITICAL

C-01	Balancer V3 Static Fee Exploit via Mixed Add/Remove Unlock Session	06
------	--	----

FINDINGS - HIGH

H-01	ltRedeemMultiAsset Arithmetic Underflow via Dust Idle Shares	07
H-02	Hook-Triggered Reinvestment Imposes Static Fee on External Removers	10
H-03	previewSyncTrancheAccounting LPT Branch Overvalues Effective NAV	10
H-04	LPT Share Price Inflation Attack at Zero Total Supply	11
H-05	LT Protocol Fee Minted Against Idle ST Shares Permanently Dilutes convertToAssets When Reinvestment Fails	12
H-06	YDM Time-Weighted Average Manipulation Converts Winning JT Long into Forced Liquidation in Uncorrelated Markets	12
H-07	_convertToShares Clamp Triggers on Deposit Size Rather Than Per-Share Price, Causing Value Donation in Normal Markets and Unbounded Share Inflation Post-Slash	12

FINDINGS - MEDIUM

M-01	JT Free Option to Exit and Keep JT Rewards After Slashing	11
M-02	_depositSharesReference Queries LPT Rate Before Sync	12
M-03	setMinCoverage Lacks Post-Update Safety Check	12
M-04	Drawdown-Recovery Cycle Re-Leverages JT Coverage Capital for ST Gain Without Compensation	13
M-05	Liquidation State Death Spiral - Irrecoverable Market Drain	08
M-06	_deriveTrancheAssetClaims Omits Pending Liquidity Premium in LPT stShares	09
M-07	Split-Execution Attack on Bootstrap Deposit Reference Causes Victim to Forfeit ~100% of Deposit Value	12

FINDINGS - LOW

L-01	Systematic Rounding Bias in _attributeDeltaToClaimOnRawNAV	13
L-02	ADMIN_MARKET_OPS_ROLE Conflates Operational and Security-Sensitive Selectors	14
L-03	_scaleAssetClaims nav Missing VIRTUAL_VALUE Addend	14
L-04	Entry Point NAV-Based Yield Forfeiture Asymmetrically Penalizes ST Depositors and Incompletely Captures JT/LPT Premium Yield	16

L-05	setLiquidationCoverageUtilization Post-Op Guard Uses == Instead of <=	16
L-06	ItRedeemMultiAsset Bypasses Receiver Blacklist Check When stSharesWithdrawn Is Zero	16
L-07	paysYieldFees: false Makes setPoolCreatorYieldFeePercentage Permanently Inert	16
L-08	JT_DEPOSIT Does Not Adjust jtCoverageImpermanentLoss	09

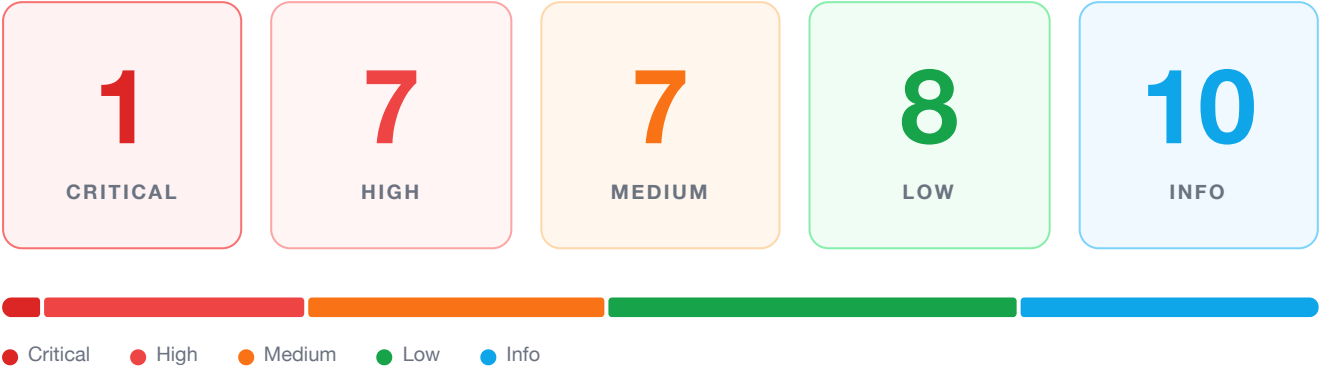
FINDINGS - INFORMATIONAL

I-01	Coinvested Denominator Always Equals stEffectiveNAV - Misleading Comment	14
I-02	YDM Market-Specific Salt Defeats Cross-Market Sharing	15
I-03	Unverified Selector Hash in Fee Controller Binding	15
I-04	Dead Code in _withdrawAssets for Non-Identical ST/JT Assets	15
I-05	Floor Rounding in _convertToValue Understates Input NAV, Weakening Slippage Protection in attemptLiquidityPremiumReinvestment	16
I-06	Partial Execution Rounding Allocates Excess Share Reference to Current Call, Enabling Incremental Fee Avoidance	16
I-07	mintLiquidityPremiumShares Accepts an Open Recipient Address Despite an Invariant Requiring Kernel-Only Custody	16
I-08	JT Locked in FIXED_TERM Observation Period When stEffectiveNAV Is Zero	17
I-09	disableUnbalancedLiquidity Silently Breaks Premium Reinvestment	16
I-10	Asset Mismatch in _cancelDepositRequest Enables EntryPoint Drain via Config Update	09

SECTION 3

Findings Overview

FINDINGS BY SEVERITY



REMEDIATION STATUS

SEVERITY	TOTAL	ACKNOWLEDGED	FIXED
Critical	1	1	1
High	7	7	7
Medium	7	7	7
Low	8	8	8
Informational	10	10	10
Total	33	33	33

SECTION 4

Protocol Overview

Royco Day is a yield-tranching protocol that splits the returns of any yield-bearing position into three risk-return profiles. All capital is coinvested into a single underlying yield source; the accountant and kernel then enforce economic rights via effective NAV accounting.

- **Senior Tranche (ST):** Capital-protected. Transfers downside to JT in exchange for a share of yield (risk premium) and liquidity (liquidity premium).
- **Junior Tranche (JT):** First-loss. Absorbs ST losses up to full coverage. Earns the risk premium from ST yield and can recover impermanent losses via future ST appreciation.
- **Liquidity Provider Tranche (LPT):** Market-making capital deployed into a Balancer V3 GyroECLP pool. Earns the liquidity premium, minted as ST shares and reinvested into the pool.

Key mechanisms include: a coverage utilization ratio (enforces JT buffer), a liquidity utilization ratio (enforces LPT depth), an adaptive/static Yield Distribution Model (YDM) curve, an oracle-gated async entry point with yield forfeiture, a self-liquidation bonus for ST in distress, FIXED_TERM/PERPETUAL market state transitions, and a Chainlink-based rate provider for the Balancer pool.

SECTION 5

Audit Scope & Methodology

Scope: <https://github.com/roycoprotocol/royco-day> — everything under `src/`

Latest Commit Reviewed: `d724c652236481fe96b5b481a8a2b16212fa0c90`

The audit combined manual code review, invariant analysis, economic attack modeling, and trace-level verification against specific commit hashes. Each finding was verified by fetching and inspecting the exact code at the referenced commit before documentation. The audit covered access control, arithmetic correctness, economic invariants, oracle manipulation, reentrancy, integration assumptions (Balancer V3), and game-theoretic attack vectors.

SECTION 6

Audit Findings

C-01 **Balancer V3 Static Fee Exploit via Mixed Add/Remove Unlock Session**

CRITICAL

LOCATION

BalancerV3VenueLogic.sol:148-158

COMMIT

72599853

DESCRIPTION

Balancer V3 charges a static swap fee on `removeLiquidity` calls that occur in the same unlock session as a prior `addLiquidity` call (enforced via a transient `addLiquidityCalledFlag`).

`removeBalancerV3Liquidity` executes the proportional BPT burn with no check on this flag. An attacker can: (0) take a flashloan outside of the Balancer protocol; (1) open an unlock and `addLiquidity` - setting the flag and acquiring a large BPT position; (2) trigger protocol LPT redemptions within the same session, each incurring the static fee; (3) collect the fees distributed to their BPT position; (4) open a new unlock to withdraw and repay the flash loan. The attack is repeatable on any block.

IMPACT

LPT redeemers are forced to pay unexpected static swap fees that accrue to the attacker's flash-added position. Protocol LPT exit efficiency is degraded and continuous value extraction is possible with zero net capital cost beyond gas and flash loan fees.

RECOMMENDED FIX

Block removal when vault is already unlocked.

MITIGATION REVIEW

Fixed

H-01 **ltRedeemMultiAsset Arithmetic Underflow via Dust Idle Shares**

HIGH

LOCATION

SelfLiquidationLogic.sol:60-62 . RoycoDayAccountant.sol:266

COMMIT

04a8c182

DESCRIPTION

When the market is in liquidation state and `ltOwnedSeniorTrancheShares` is dust (e.g., 1 share), a `ltRedeemMultiAsset` call can reach a state where `stSelfLiquidationBonusNAV > 0` but `totalSTAndJTRedemptionNAV == 0`. This occurs because `_computeMaxCoverageUtilizationNeutralBonus` gates only on `nav == 0` (not on asset amounts), so a single-share claim with zero redeemable assets still produces a non-zero bonus NAV. The accountant then computes `stEffectiveNAV -= (0 - bonusNAV)`, triggering an arithmetic underflow and reverting the entire transaction. Additionally, when the ST redeemer's proportional claim rounds to zero ST assets (dust share count relative to total supply), the self-liquidation bonus computation returns a non-zero NAV bonus with a zero-asset settlement, causing the post-op accountant check to fail and permanently reverting the redemption for that position.

IMPACT

Any `ltRedeemMultiAsset` call landing in this state permanently reverts, bricking the multi-asset redemption path during the liquidation regime - precisely when LPT holders most urgently need to exit. The condition is reproducible whenever the GyroECLP pool skews toward the quote side and residual dust idle ST shares remain. Additionally, ST holders with dust positions in liquidation state cannot redeem, even though the market should be incentivizing exits; this DoS is bounded to dust positions but can be manufactured by an attacker who controls share distribution.

RECOMMENDED FIX

Calculate bonus NAV by bonus assets.

MITIGATION REVIEW

Fixed

H-02 Hook-Triggered Reinvestment Imposes Static Fee on External Removers

HIGH

LOCATION

RoycoDayBalancerV3Hooks.sol:101-117

COMMIT

72599853

DESCRIPTION

The `onBeforeRemoveLiquidity` hook fires `syncTrancheAccounting` for every non-kernel `removeLiquidity` call. This sync invokes `attemptLiquidityPremiumReinvestment` whenever idle ST shares exist, executing a `vault.addLiquidity` that sets Balancer V3's `addLiquidityCalledFlag = true`. Any subsequent `removeLiquidity` in the same unlock session (including the external user's own remove) is then charged the static swap fee. The user had no way to anticipate or prevent the hook firing the reinvestment add within their unlock session.

IMPACT

Third-party pool participants removing liquidity are silently charged a static swap fee whenever the kernel holds pending premium shares, without consent or prior notice. The fee accrues partially back to the kernel (as a BPT holder), making the protocol an inadvertent beneficiary of fees extracted from external participants and degrading the pool's competitiveness.

RECOMMENDED FIX

Don't do reinvestments in hooks, or remove hooks.

MITIGATION REVIEW

Fixed

H-03 **previewSyncTrancheAccounting LPT Branch Overvalues Effective NAV**

HIGH

LOCATION

AccountingSyncLogic.sol:127-136

COMMIT

72599853

DESCRIPTION

In the LPT branch of `previewSyncTrancheAccountingFor`, newly accrued liquidity-premium ST shares are valued as idle ST at the ST share rate - no reinvestment slippage is modeled. At execution, `attemptLiquidityPremiumReinvestment` converts these shares to BPT at a loss bounded by `maxReinvestmentSlippageWAD`. The resulting overstatement of `claims.stShares` and `claims.nav` propagates into `lptMaxWithdrawableMultiAsset`, inflating the liquidity requirement reduction calculation and causing `maxRedeemMultiAsset` to return a share count too high for actual execution. The `type(uint256).max` sentinel input - which uses this bound - always reverts on the post-op liquidity condition whenever reinvestment succeeds.

IMPACT

The `type(uint256).max` redemption path permanently fails when reinvestment is pending and expected to succeed. LPT holders cannot use the convenient maximum sentinel and must manually calculate a safe share count, removing the key usability guarantee of the maximum-redeem path.

RECOMMENDED FIX

Remove reinvestment from pre-op sync.

MITIGATION REVIEW

Fixed

H-04 LPT Share Price Inflation Attack at Zero Total Supply

HIGH

LOCATION

ValuationLogic.sol:119-121 . RoycoLiquidityTranche.sol:75 . RedemptionLogic.sol:138-139

COMMIT

76ef6bca

DESCRIPTION

`_convertToShares` returns shares 1:1 with the deposited NAV value whenever `_totalSupply == 0`, ignoring `_totalValue` entirely. This branch is correct for a cold-start deployment where no value exists yet. LPT can reach `totalSupply == 0` with a non-zero effective NAV: Balancer swap fees accumulate passively as BPT inside `ltOwnedYieldBearingAssets`, and liquidity premium ST shares accrue in `ltOwnedSeniorTrancheShares` - neither is cleared when the last LPT holder exits. An attacker who observes this state deposits 1 wei, receives shares sized against only their dust deposit, and becomes the sole LPT holder capturing the entire accumulated BPT fee balance and all pending ST premium shares.

The zero-supply condition is directly reachable through the liquidation exit path: in `ltRedeem`, when `coverageUtil >= liquidationUtil`, the flag `_enforceCoverageAndLiquidityRequirements` is set to false, allowing the last LPT holder to fully withdraw its entire inventory even during the market's most stressed state. The exit clears `totalSupply` to zero while `ltOwnedYieldBearingAssets` and `ltOwnedSeniorTrancheShares` remain behind. The two issues chain: liquidation enables a full LPT exit, the full exit creates the zero-supply state, and the zero-supply state enables the inflation attack.

IMPACT

Classic inflation attack where donation is premium paid for LPT by ST.

RECOMMENDED FIX

Seed the pool at the start and burn the shares.

MITIGATION REVIEW

Fixed

H-05

LT Protocol Fee Minted Against Idle ST Shares Permanently Dilutes convertToAssets When Reinvestment Fails

HIGH

LOCATION

FeeAndLiquidityPremiumLogic.sol:67-75

COMMIT

952d2296

DESCRIPTION

The LT protocol fee shares are minted against `_getLiquidityTrancheEffectiveNAV` as the denominator, which includes both `ltRawNAV` (the GyroECLP AMM position) and the value of `ltOwnedSeniorTrancheShares` (idle, un-reinvested ST premium shares held by the kernel). However, the LT vault's `convertToAssets` tracks only `ltRawNAV`; the idle ST shares are not part of `totalAssets()`. When `attemptLiquidityPremiumReinvestment` partially or fully fails - e.g., because the GyroECLP pool cannot absorb more inventory - the earned premium remains as idle ST shares: it never enters `ltRawNAV` and never appears in `convertToAssets`. The fee mint still issues shares sized against the full `ltEffectiveNAV`, increasing total LT supply without any corresponding increase in `ltRawNAV`. The result is a permanent decrease in `convertToAssets` per LT share for that sync.

IMPACT

In any cycle where GyroECLP reinvestment is constrained, the LT `convertToAssets` is permanently decremented by the fee dilution with no offsetting income. Repeated cycles compound the effect. Any integration, oracle, or accounting that relies on LT `convertToAssets` being non-decreasing - including the protocol's own cached ST share rate path - observes a monotonically declining LT share rate despite the LT earning its full liquidity premium.

RECOMMENDED FIX

Take fees from LPT by paying the fee from the premium paid to LPT in ST shares.

MITIGATION REVIEW

Fixed

H-06

YDM Time-Weighted Average Manipulation Converts Winning JT Long into Forced Liquidation in Uncorrelated Markets

HIGH

LOCATION

RoycoDayAccountant.sol:694-723 · RoycoDayAccountant.sol:616

COMMIT

539418f8

DESCRIPTION

When `beta = 0` (ST and JT assets are uncorrelated), there is no natural offset between ST appreciation and the JT coverage position. An attacker can exploit the JT YDM's time-weighted average yield share computation to force this uncorrelated gain asymmetry into a liquidation event via the following sequence: (1) while JT utilization is very low, queue a large exit, locking the request-time NAV at a favorable rate; (2) wait for an oracle update; (3) just before the next oracle update, execute the redemption - this pushes JT utilization from near-zero to 100%, but because the time-weighted average is computed over the full observation window during which utilization was low, the resulting weighted average yield share is minimal; (4) with the yield share now anchored low, any large ST yield event distributes almost all yield to ST, causing ST NAV to outpace JT NAV on a percentage basis; (5) the resulting NAV divergence raises JT coverage utilization beyond the liquidation threshold, triggering JT liquidation and routing the self-liquidation bonus to ST holders.

IMPACT

JT's economic premise is a leveraged long on the ST asset - JT holders bear downside to earn the upside when ST appreciates. In this attack, ST appreciates strongly and JT is nevertheless liquidated, with its collateral redistributed as a bonus to ST holders. A JT holder who was correct in their directional bet loses funds precisely because they were right. This is a fundamental violation of the market's guarantee: being too correct on the long side should never result in a loss.

RECOMMENDED FIX

Do not allow uncorrelated assets (`beta = 0`) to be configured across the ST and JT tranches.

MITIGATION REVIEW

Fixed

H-07 **`_convertToShares` Clamp Triggers on Deposit Size Rather Than Per-Share Price, Causing Value Donation in Normal Markets and Unbounded Share Inflation Post-Slash**

HIGH

LOCATION

ValuationLogic.sol:127-130

COMMIT

539418f8

DESCRIPTION

The `_convertToShares` function in `ValuationLogic` (lines 127-130) applies a clamp intended to bound the number of new shares minted when the existing per-share value becomes extremely low after JT slashing. The clamp condition checks whether the deposited amount exceeds `totalValue * 1e12` - i.e., whether the ratio of the incoming deposit to the residual pool value is greater than `1e12`. Because this check operates on raw token amounts rather than per-share prices, it fires based on size disparity between the deposit and the pool, not on whether the per-share value is actually skewed.

Two failure modes arise. First, in a healthy market where the share-to-NAV ratio is still 1:1 but `totalValue` happens to be small (e.g., early in a market's life), a sufficiently large deposit can exceed `totalValue * 1e12` and trigger the clamp. The depositor receives a capped share count and silently donates the excess value to existing holders - an unintended wealth transfer. Second, after JT slashing drives residual assets to zero, a virtual NAV of 1 per existing share is substituted to prevent a division-by-zero revert. Under this convention a deposit of `N` assets mints roughly `N * totalShares` new shares. The clamp is supposed to cap this but its condition is evaluated against the already-near-zero `totalValue`, so it fires immediately and does not prevent the unbounded inflation - successive deposits after each slash can multiply `totalSupply` by an unbounded factor, eventually causing arithmetic overflows and DoSing the tranche.

IMPACT

In normal operating conditions the clamp silently transfers depositor value to existing holders, constituting an unintended economic penalty that worsens as pool size decreases. After a JT slashing event that drives residual assets to near-zero, uncapped share inflation compounds with each subsequent deposit. Repeated slashing-then-deposit cycles can grow `totalSupply` to values that cause overflow reverts in downstream arithmetic, permanently DoSing the affected tranche.

RECOMMENDED FIX

Change the clamp condition to trigger only when the per-share value is skewed below a meaningful threshold (e.g., `totalValue / totalSupply < 1e-12` NAV units) rather than when the deposit-to-value ratio exceeds a constant. Apply the cap solely as an upper bound on shares minted - preventing inflation in genuinely skewed states without penalising depositors or donating their value in normal market conditions.

MITIGATION REVIEW

Fixed

M-01 JT Free Option to Exit and Keep JT Rewards After Slashing

MEDIUM

LOCATION

RoycoDayEntryPoint.sol:201 · RoycoDayEntryPoint.sol:655

COMMIT

952d2296

DESCRIPTION

A sophisticated JT holder can request to exit before a JT slashing event, fixing the request-time NAV at the pre-slash rate. While sitting in the redemption queue the position continues to accrue JT yield; the NAV-at-request-time lock was designed to prevent this carry, but the queue window grants it freely. This gives JT holders a free option: when the next slashing event occurs they can redeem at pre-slash pricing while having collected the upside during the delay.

IMPACT

Informed JT holders can systematically time exits to avoid loss absorption, undermining the first-loss guarantees that ST holders rely on. The attack is conditional on oracle update predictability but is economically rational whenever the expected loss exceeds the oracle delay cost.

RECOMMENDED FIX

Add an expiration on the redeem request.

MITIGATION REVIEW

Fixed

M-02 **_depositSharesReference Queries LPT Rate Before Sync**

MEDIUM

LOCATION

RoycoDayEntryPoint.sol:671-676

COMMIT

72599853

DESCRIPTION

In `_depositSharesReference`, `convertLPTAssetsToValue` is called before `syncTrancheAccountingFor`. The sync may execute `attemptLiquidityPremiumReinvestment`, which adds assets to the GyroECLP pool, changing the BPT price. The `depositValue` numerator therefore uses the pre-sync BPT conversion rate while the `trancheClaims.nav` denominator reflects the post-sync pool state. The resulting `equivalentSharesAtRequestTime` reference is systematically inconsistent - understated when reinvestment increases the per-BPT value - causing the yield forfeiture to over-charge LPT depositors at execution time.

IMPACT

LPT depositors who submit requests at blocks where a liquidity premium reinvestment occurs during sync are charged yield forfeiture fees on value they did not actually earn. The discrepancy grows with the size of the pending premium reinvestment.

RECOMMENDED FIX

Remove reinvestment from sync, or sync at the start of `depositRequest`.

MITIGATION REVIEW

Fixed

M-03 **setMinCoverage Lacks Post-Update Safety Check**

MEDIUM

LOCATION

RoycoDayAccountant.sol:746-766

COMMIT

76ef6bca

DESCRIPTION

`setMinCoverage` validates only that `_minCoverageWAD < WAD`, with no check that the post-update coverage utilization remains below the liquidation threshold. Since `coverageUtilizationWAD = collateralNAV x minCoverageWAD / jtEffectiveNAV`, raising `minCoverageWAD` increases utilization proportionally. A misconfigured or malicious parameter change can instantly push a healthy market into the liquidation regime, enabling the death spiral described in [Liquidation State Death Spiral - Irrecoverable Market Drain](#). The symmetric gap exists in `setLiquidationCoverageUtilization`, where lowering the threshold below current utilization silently enables self-liquidation bonuses on a solvent market.

IMPACT

An admin action - erroneous or malicious - can instantaneously trigger the liquidation regime without any collateral deterioration, exposing JT holders to the death spiral with no on-chain warning and no opportunity to withdraw before the parameter takes effect.

RECOMMENDED FIX

Check that utilization decreases or is less than WAD.

MITIGATION REVIEW

Fixed

M-04

Drawdown-Recovery Cycle Re-Leverages JT Coverage Capital for ST Gain Without Compensation

MEDIUM

LOCATION

RoycoDayAccountant.sol:447 · RoycoDayAccountant.sol:466-483

COMMIT

1c4df3d8

DESCRIPTION

After a drawdown-recovery cycle, JT receives two things: its own yield for the period - already reduced because JT's effective NAV was diminished during coverage - and a nominal reimbursement from ST for the coverage principal. However, the excess yield that ST earned by deploying the coverage capital during the drawdown period is attributed entirely to ST. ST effectively holds JT's capital interest-free: it pays back only the nominal amount while retaining all appreciation generated on that capital. This excess yield was produced using JT's balance sheet and should, at minimum prior to risk premium distribution, flow proportionally back to JT.

Concrete example (price path 1 → 0.8 → 2):

State	JT assets	JT NAV	JT IL	ST assets	ST NAV
Price = 1 (start)	50	50	0	100	100
Price drops to 0.8	25	20	30	125	100
Price recovers to 2	25	50	0	125	250
Direct 1 → 2 (no drawdown)	50	100	0	100	200

JT ends the round-trip with NAV = 50 - identical to its starting value despite the asset price doubling. ST ends with NAV = 250 instead of the 200 it would have earned on a direct 1 → 2 move. The extra 50 is the appreciation ST captured by deploying JT's 25 assets as coverage during the recovery phase, paid back to JT only at nominal cost (IL cleared to zero) while ST retains all the upside generated on that capital.

IMPACT

ST systematically captures yield above its economic entitlement at JT's expense in any market experiencing volatility. The IL mechanism - designed to ensure JT recoups its coverage cost - is bypassed whenever JT's own recovery gain is sufficient to clear the IL ledger.

RECOMMENDED FIX

First restore IL to JT and only then distribute yield between tranches.

MITIGATION REVIEW

Fixed

M-05 **Liquidation State Death Spiral - Irrecoverable Market Drain**

MEDIUM

LOCATION

SelfLiquidationLogic.sol:120

COMMIT

952d2296

DESCRIPTION

The self-liquidation bonus is designed to incentivize ST to exit and deleverage the market when coverage utilization breaches the liquidation threshold. However, each ST exit paired with a coverage-utilization-neutral bonus leaves `coverageUtilizationWAD` exactly unchanged by construction. No individual ST exit reduces coverage utilization through the bonus path. Recovery therefore requires net new JT capital - but JT holders face a dominant-strategy disincentive to deposit: any JT deposit that leaves the market in the liquidation zone is immediately front-run by ST redemptions that extract the deposit as bonus, returning utilization to its prior level.

IMPACT

Any market that enters the liquidation state under adverse price conditions faces a one-way drain. ST exits continuously; JT effective NAV trends toward zero; coverage utilization diverges upward. The only escape is an exogenous collateral price appreciation large enough to single-handedly drop utilization below the threshold. Without such appreciation, the market drains irrecoverably regardless of admin intervention or JT rescue attempts.

RECOMMENDED FIX

Block JT deposits if the coverage utilization after the deposit is greater than or equal to the liquidation utilization.

MITIGATION REVIEW

Fixed

M-06

`_deriveTrancheAssetClaims` Omits Pending Liquidity Premium in LPT `stShares`

MEDIUM

LOCATION

`TrancheClaimsLogic.sol:62-68`

COMMIT

952d2296

DESCRIPTION

In the LPT branch of `_deriveTrancheAssetClaims`, `claims.stShares` is set to `$.ltOwnedSeniorTrancheShares` - the pre-mint value before the liquidity premium ST shares are minted. The execution path mints the premium shares before calling `_deriveTrancheAssetClaims` (so the execution value is correct), but the preview path in `previewSyncTrancheAccounting` also calls this function, obtaining a `claims.stShares` that does not include the newly accrued premium. This causes the preview to undercount LPT's idle ST share claim, producing an understated LPT effective NAV and an inconsistent view of the LPT's redeemable assets.

IMPACT

Preview functions used by integrators, front-ends, and the entry point's yield forfeiture logic return an LPT NAV that does not account for pending premium - systematically understating LPT redemption values and affecting yield forfeiture calculations for LPT depositors.

RECOMMENDED FIX

Add new minted premium shares to `claims.stShares`.

MITIGATION REVIEW

Fixed

M-07

Split-Execution Attack on Bootstrap Deposit Reference Causes Victim to Forfeit ~100% of Deposit Value

MEDIUM

LOCATION

RoycoDayEntryPoint.sol:678

COMMIT

650d9d3d

CONTEXT

This finding was introduced as a direct consequence of the fix applied to H-04. The H-04 mitigation adopted OpenZeppelin's virtual shares mechanism, seeding every new tranche with 1e6 virtual shares on initialization to prevent first-depositor exchange rate manipulation. While that fix resolves the H-04 attack surface, it creates a new one: the 1e6 virtual share offset embedded in the ERC4626 formula $(supply + 1e6) \times assets / (totalValue + 1)$ causes the yield-forfeiture logic to interpret the resulting share-count discrepancy as a NAV appreciation event eligible for protocol fees, stripping the victim of nearly all deposited value when the bootstrap deposit reference is split across multiple executions.

DESCRIPTION

`_depositSharesReference` is allowed to be partially executed across multiple calls. When a victim queues a large deposit while the tranche has zero total supply, the NAV-at-request-time is locked at the 1:1 bootstrap rate. After the queue delay passes, an attacker can execute the request in two steps within a single transaction: first call `executeDeposit(victim, nonce, 1)` which mints 1 share at 1:1, seeding the tranche with `totalSupply = 1` and `totalValue = 1`; then call `executeDeposit(victim, nonce, type(uint256).max)` for the remaining assets. The second call now uses the OpenZeppelin ERC4626 formula $(supply + 1e6) \times assets / (totalValue + 1)$, producing an execution share count vastly larger than the 1:1 reference locked at request time. The yield-forfeiture logic in `_depositWithYieldForfeiture` computes $protocolFeeShares = userTrancheShares \times (navAtExecutionTime - navAtRequestTime) / navAtExecutionTime$ and transfers that delta - nearly all minted shares - to the protocol fee recipient, leaving the victim with almost nothing.

IMPACT

Victim loses approximately 99.9998% of their deposited asset value to the protocol fee recipient. The attack is permissionless, requires no capital beyond gas, and is triggered by any state in which the victim's own queued deposit seeds the tranche from zero supply. Any deposit that enters the queue during an empty-tranche state is vulnerable until the tranche is seeded by an independent deposit.

RECOMMENDED FIX

Make mint consistent and continuous - no special cases in the bootstrap rate reference.

MITIGATION REVIEW

Fixed

I-01 Systematic Rounding Bias in `_attributeDeltaToClaimOnRawNAV`

LOW

LOCATION

RoycoDayAccountant.sol:963-980

COMMIT

952d2296

DESCRIPTION

Always floors the absolute magnitude regardless of sign, routing consistent rounding drift in favor of seniors on losses and juniors on gains, accumulating across syncs.

IMPACT

Dust-level but directional wealth transfer between tranches that compounds over many sync cycles in active markets.

MITIGATION REVIEW

Fixed

I-02 ADMIN_MARKET_OPS_ROLE Conflates Operational and Security-Sensitive Selectors

LOW

LOCATION

BalancerV3_GyroECLP_LT_DeploymentTemplate.sol:540-543

COMMIT

76ef6bca

DESCRIPTION

`reinvestLiquidityPremium` and `setRoycoBlacklist` share a single `ADMIN_MARKET_OPS_ROLE`, granting operational operators the ability to modify the sanctions list or vice versa.

IMPACT

Violates least-privilege. A key compromise for routine operations also grants sanctions-list write access, a compliance-sensitive capability that warrants a stricter role boundary.

MITIGATION REVIEW

Fixed

I-03 **_scaleAssetClaims nav Missing VIRTUAL_VALUE Addend**

LOW

LOCATION

TrancheClaimsLogic.sol:118

COMMIT

72599853

DESCRIPTION

When `_includeVirtualShares` is true the denominator is inflated by `VIRTUAL_SHARES` but the nav numerator is not correspondingly inflated by `VIRTUAL_VALUE`, producing a systematically understated nav in the `_redemptionValueReference` path.

IMPACT

Understated redemption value reference at the virtual-shares boundary; practically negligible in well-funded pools but creates an accounting inconsistency that compounds if the virtual balance is a significant fraction of total NAV.

MITIGATION REVIEW

Fixed

I-04

Entry Point NAV-Based Yield Forfeiture Asymmetrically Penalizes ST Depositors and Incompletely Captures JT/LPT Premium Yield

LOW

LOCATION

RoycoDayEntryPoint.sol:104 · RoycoDayEntryPoint.sol:630

COMMIT

3401dd0f

DESCRIPTION

On ST deposit request, the depositor is punished for no reason - gets zero yield even when all users inside and outside of the protocol receive yield.

IMPACT

This will prevent users from depositing to the protocol.

MITIGATION REVIEW

Fixed

L-05 **setLiquidationCoverageUtilization Post-Op Guard Uses == Instead of <=**

LOW

LOCATION

RoycoDayAccountant.sol:51

COMMIT

2b9d2a1f

DESCRIPTION

The post-operation guard in `setLiquidationCoverageUtilization` asserts `preOp.coverageLiquidationUtilizationWAD == postOp.coverageLiquidationUtilizationWAD`. This check only passes when the threshold is exactly unchanged, making the setter non-functional for any actual threshold change. The intended invariant is that the threshold may only be raised - making liquidation harder to trigger - expressed as `preOp.coverageLiquidationUtilizationWAD <= postOp.coverageLiquidationUtilizationWAD`. With the `==` operator, any governance call to update the threshold reverts at the post-op check when the market is in distress - not unconditionally, but specifically under distressed market conditions where the post-op state diverges from the pre-op snapshot.

IMPACT

The liquidation coverage utilization threshold is permanently locked to its initialized value - governance cannot raise it even when market conditions warrant a more conservative threshold.

MITIGATION REVIEW

Fixed

L-06

ltRedeemMultiAsset Bypasses Receiver Blacklist Check When stSharesWithdrawn Is Zero

LOW

LOCATION

RoycoDayEntryPoint.sol:241, RedemptionLogic.sol:160-211

COMMIT

3401dd0f

DESCRIPTION

`executeRedemption` in `RoycoDayEntryPoint.sol` does not directly screen the receiver against the blacklist, delegating that responsibility to the kernel's redeem functions. This is correct for the ST, JT, and LT in-kind paths, where the kernel's `stRedeem` / `jtRedeem` / `ltRedeem` each enforce `BlacklistLogic._enforceNotBlacklisted` on the receiver before any asset transfer.

In the `ltRedeemMultiAsset` flow there are two legs that could screen the receiver. The first leg calls `removeLiquidity` with the receiver set to the kernel itself, so no receiver blacklist check occurs there. The second leg redeems the venue-returned ST shares and transfers ST and JT assets directly to the receiver - but this leg is conditioned on `stSharesWithdrawn != 0`. When the venue removal returns zero ST shares (e.g., the pool composition contains no ST at the time of removal), the ST-leg is skipped entirely and the receiver is never screened. Assets from the liquidity removal (quote tokens) reach the receiver without any blacklist enforcement.

IMPACT

A blacklisted address can receive assets through the `ltRedeemMultiAsset` path when the Balancer venue returns zero ST shares on liquidity removal, circumventing the protocol's blacklist enforcement.

MITIGATION REVIEW

Fixed

L-07

paysYieldFees: false Makes setPoolCreatorYieldFeePercentage Permanently Inert

LOW

LOCATION

BalancerV3_GyroECLP_LT_DeploymentTemplate.sol:442 .
BalancerV3_GyroECLP_LT_DeploymentTemplate.sol:597-604

COMMIT

952d2296

DESCRIPTION

`_buildTokenConfig` hardcodes `paysYieldFees: false` for every token registered in the Gyro ECLP pool, regardless of whether the token is the ST share (a rate-bearing `WITH_RATE` token whose rate rises as senior NAV appreciates) or the quote asset. In Balancer V3 this flag is stored immutably per token at pool registration time and controls whether the vault accumulates yield fees on rate appreciation for that token. Because both tokens are registered with the flag unset, the vault never enters the yield-fee accrual path for this pool.

`_balancerProtocolFeeControllerBinding` binds `IPoolFeeController.setPoolCreatorYieldFeePercentage.selector` and `withdrawPoolCreatorFees` to `ADMIN_BALANCER_POOL_MANAGER_ROLE`, but neither function has any effect on a pool whose tokens were registered with `paysYieldFees: false` - the stored percentage is irrelevant because no fees ever accumulate.

IMPACT

The protocol cannot collect Balancer yield fees on ST share appreciation flowing through the pool, even if the admin later configures a non-zero pool creator yield fee percentage. The two role bindings granted to `ADMIN_BALANCER_POOL_MANAGER_ROLE` for yield-fee management are permanently dead for every market deployed from this template, and there is no on-chain mechanism to override the per-token flag after registration.

MITIGATION REVIEW

Fixed

I-08 JT_DEPOSIT Does Not Adjust jtCoverageImpermanentLoss

LOW

LOCATION

RoycoDayAccountant.sol:243-246

COMMIT

76ef6bca

DESCRIPTION

When a new JT deposit occurs while `jtCoverageImpermanentLoss` (IL) is non-zero, the protocol faces an irresolvable accounting dilemma. The IL represents JT's accumulated recovery claim on future ST appreciation earned by absorbing prior coverage losses. If the IL is scaled up to reflect the new depositor's proportional share, value is transferred away from ST holders who are entitled to that appreciation. If the IL is left unchanged - the current behavior - the new depositor is implicitly granted a fractional IL recovery claim on losses they never bore, diluting the recovery rights of existing JT holders.

IMPACT

New JT depositors acquire IL recovery rights on losses they did not cover, transferring future ST appreciation away from existing JT holders who bore the original loss. The degree of theft scales with the size of the deposit relative to the existing JT pool and the magnitude of the outstanding IL.

MITIGATION REVIEW

Fixed

I-01 Coinvested Denominator Always Equals stEffectiveNAV - Misleading Comment

INFORMATIONAL

LOCATION

SelfLiquidationLogic.sol:124

COMMIT

952d2296

DESCRIPTION

The documented derivation of `_computeMaxCoverageUtilizationNeutralBonus` uses a "coinvested" denominator that reduces by NAV conservation to exactly `stEffectiveNAV`. The subtraction form in the comment is misleading and could confuse future maintainers into thinking the denominator is distinct.

IMPACT

No runtime impact. Documentation clarity issue only.

MITIGATION REVIEW

Fixed

I-02 YDM Market-Specific Salt Defeats Cross-Market Sharing

INFORMATIONAL

LOCATION

BaseDeploymentTemplate.sol:215

COMMIT

76ef6bca

DESCRIPTION

`_deployYDM` uses a market-scoped salt that guarantees a fresh YDM contract per market, negating the code path designed to allow YDM reuse across markets with identical parameters.

IMPACT

Intended gas savings from YDM reuse are never realised; dead code path may mislead future developers into assuming sharing works.

MITIGATION REVIEW

Fixed

I-03 Unverified Selector Hash in Fee Controller Binding

INFORMATIONAL

LOCATION

BalancerV3_GyroECLP_LT_DeploymentTemplate.sol:609

COMMIT

04a8c182

DESCRIPTION

`_balancerProtocolFeeControllerBinding` computes the `withdrawPoolCreatorFees` selector via raw `keccak256` of a string literal rather than `abi.encodeCall` or an interface cast, silently producing the wrong selector if the function signature changes upstream.

IMPACT

No immediate runtime risk given the current Balancer V3 version, but creates a latent break on upstream API changes with no compile-time guard.

MITIGATION REVIEW

Fixed

I-04 Dead Code in `_withdrawAssets` for Non-Identical ST/JT Assets

INFORMATIONAL

LOCATION

TrancheClaimsLogic.sol:103-106

COMMIT

16d3df92

DESCRIPTION

The `else` branch in `_withdrawAssets` handling distinct ST and JT tokens is unreachable because `RoycoDayKernel` requires `stAsset == jtAsset` at construction, making the condition always false.

IMPACT

Dead code with no runtime impact; increases maintenance surface and may mislead auditors into assuming dual-asset support exists.

MITIGATION REVIEW

Fixed

I-05 Floor Rounding in `_convertToValue` Understates Input NAV, Weakening Slippage Protection in `attemptLiquidityPremiumReinvestment`

INFORMATIONAL

LOCATION

BalancerV3VenueLogic.sol L205

COMMIT

0019ec31

DESCRIPTION

`attemptLiquidityPremiumReinvestment` converts `stSharesToReinvest` to NAV units using `Math.Rounding.Floor` and passes the result to `convertValueToLPTAssets` to derive `minLPTAssetsOut`. Because `minLPTAssetsOut` is a minimum output floor, the input valuation should round `Ceil` - understating it weakens the slippage bound.

IMPACT

The effective slippage accepted exceeds `_maxReinvestmentSlippageWAD` by the rounding error.

MITIGATION REVIEW

Fixed

I-06

Partial Execution Rounding Allocates Excess Share Reference to Current Call, Enabling Incremental Fee Avoidance

INFORMATIONAL

LOCATION

RoycoDayEntryPoint.sol L418-L420

COMMIT

a150cbad

DESCRIPTION

When `_executeDeposit` handles a partial fill, the stored `equivalentSharesAtRequestTime` for the remaining assets is scaled with `Floor` rounding (L418). The current execution's reference is the residual - total minus the floor-rounded remainder (L420) - which rounds up by construction. The executing call therefore receives a share reference exceeding its proportional entitlement by up to 1 share. Since `_depositWithShareForfeiture` forfeits `max(0, sharesAtExecutionTime - equivalentSharesAtRequestTime)` to the protocol, the inflated reference directly reduces the fee. Because `_assetsToDeposit` is caller-controlled, a user can split a single request into arbitrarily many partial calls and accumulate one rounding credit per execution. Each partial call also defers consumption of the remaining request, allowing the escrowed assets to continue accruing base yield while the total forfeiture owed is progressively reduced.

IMPACT

The NAV-based forfeiture mechanism is bypassable through partial self-execution. A depositor who splits a request into N calls recovers up to N shares of forfeiture that would otherwise accrue to the protocol, bounded only by the total `equivalentSharesAtRequestTime` of the request.

MITIGATION REVIEW

Fixed

I-07

mintLiquidityPremiumShares Accepts an Open Recipient Address Despite an Invariant Requiring Kernel-Only Custody

INFORMATIONAL

LOCATION

RoycoSeniorTranche.sol L30

COMMIT

650d9d3d

DESCRIPTION

`mintLiquidityPremiumShares` accepts a `_to` address parameter under `onlyKernel` access control. The design invariant is that the recipient must always be the kernel, which tracks the minted shares via `$.ltOwnedSeniorTrancheShares`. The tranche already holds an immutable `KERNEL` reference and the sole call site in `FeeAndLiquidityPremiumLogic` always passes `address(this)`, making `_to` redundant. Because `$.ltOwnedSeniorTrancheShares` is incremented by the caller rather than verified on the tranche side, any future kernel call path that passes a non-kernel `_to` would mint premium shares to an arbitrary address while the accounting entry still credits the kernel, permanently breaking the LT's custodied share balance without triggering a revert.

IMPACT

The function is safe in its current call site. The unnecessary `_to` parameter widens the attack surface for future kernel changes: any new code path that routes this call with a caller-supplied address silently misroutes the liquidity premium without on-chain detection.

MITIGATION REVIEW

Fixed

I-08

JT Locked in FIXED_TERM Observation Period When stEffectiveNAV Is Zero

INFORMATIONAL

LOCATION

RoycoDayAccountant.sol:547-550

COMMIT

13f528821c

DESCRIPTION

The perpetual-state guard evaluates five conditions to force a `FIXED_TERM - PERPETUAL` transition, but none of them check for `stEffectiveNAV == ZERO_NAV_UNITS`. When a loss event triggers `FIXED_TERM` before any ST capital has entered the market, `stEffectiveNAV` is zero: no ST holders are present to be protected, JT is earning no risk premium, and no ST yield exists to offset JT's impermanent loss. The observation window has no economic purpose in this state yet the guard never triggers the `FIXED_TERM - PERPETUAL` transition, keeping JT locked for the full `fixedTermDurationSeconds`.

IMPACT

JT depositors are locked in `FIXED_TERM` for the full observation window in a market where no ST capital is present. With no ST participating, JT is neither protecting any senior depositor nor receiving a risk premium for doing so.

MITIGATION REVIEW

Fixed

I-09 **disableUnbalancedLiquidity Silently Breaks Premium Reinvestment**

INFO

LOCATION

BalancerV3_GyroECLP_LT_DeploymentTemplate.sol:420-421

COMMIT

76ef6bca

DESCRIPTION

The deployment template allows `disableUnbalancedLiquidity` to be set on the GyroECLP pool without any guards. The premium reinvestment path uses a single-sided (unbalanced) add-liquidity operation to deploy idle ST shares into the pool. If `disableUnbalancedLiquidity` is enabled on the pool, every call to `attemptLiquidityPremiumReinvestment` will silently revert (the reversion is swallowed by the `tolerates-reversion` wrapper). The LPT premium accumulates indefinitely as idle ST shares, accruing at the ST share rate but never entering the pool - producing a persistent and growing deviation between preview NAV and actual deployable capital.

IMPACT

LPT's market-making inventory is permanently stunted; the idle premium shares grow without bound, distorting the LPT effective NAV and creating mispriced coverage ratios. The condition is irreversible after deployment without a full market migration.

MITIGATION REVIEW

Fixed

I-10

Asset Mismatch in `_cancelDepositRequest` Enables EntryPoint Drain via Config Update

INFO

LOCATION

RoycoDayEntryPoint.sol:559-560 . RoycoDayEntryPoint.sol:837-864

COMMIT

d0751ab4

DESCRIPTION

`_cancelDepositRequest` resolves the refund asset from the live `trancheToConfig[tranche].asset` at cancellation time rather than from a snapshot taken at deposit time. The `DepositRequest` struct records only the tranche address and asset amount - no asset address. `_modifyTrancheConfigs` re-reads `IRoycoVaultTranche(tranche).asset()` on every invocation, overwriting the stored asset for any registered tranche.

An attacker in control of the factory admin role can exploit this by deploying a tranche through a malicious (factory-enabled) template whose `asset()` return value is mutable: (1) deploy a malicious tranche whose `asset()` initially returns a worthless token and register it via the factory; (2) call `modifyTrancheConfigs` so the EntryPoint stores the low-value token as the tranche asset; (3) create large deposit requests depositing the low-value token into the EntryPoint; (4) mutate the malicious tranche's `asset()` to return a high-value token already held by the EntryPoint; (5) call `modifyTrancheConfigs` again, updating the stored asset to the high-value token; (6) cancel the deposit requests, receiving the high-value token from the EntryPoint's balance.

A broader long-term risk exists independently of active malice: a future factory upgrade containing a code mistake, or a compromised admin key at any point after deployment, can register a new template that exploits this same primitive against deposits made under the original honest factory. Because `_cancelDepositRequest` reads the live config at cancellation time with no binding to the asset recorded at deposit time, any evil template introduced months or years after a deposit was created can retroactively redirect that deposit's refund. Every pending deposit request in the EntryPoint is permanently exposed to any template the factory admin registers in the future - past deployments provide no protection.

IMPACT

Any ERC-20 token held by the EntryPoint - across all markets - can be drained. The EntryPoint aggregates escrowed deposits from all supported tranches, making it a high-value target. A compromised or malicious factory admin can execute this attack without triggering any of the kernel's guarded flows.

MITIGATION REVIEW

Fixed

SECTION 7

Disclaimer

This report was prepared by Tomer Security First based on the codebase provided by the client at the specified commits and within the defined audit scope. The findings and conclusions are based on the code reviewed during the engagement period and do not constitute an exhaustive enumeration of all possible vulnerabilities. Smart contract security audits reduce - but do not eliminate - the risk of exploits. Tomer Security First makes no representations or warranties regarding the fitness of the audited code for any particular purpose, and shall not be held liable for any losses arising from the deployment or use of the audited system.